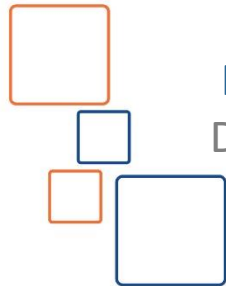


# CORE JAVA PROGRAMMING

AN INTRODUCTION TO JAVA



Presented By  
Dr. Eman Hesham, DBA.



Java™ Education  
and Technology Services



Invest In Yourself,  
**Develop** Your Career

# Online Java development Course

- ▶ <https://maharatech.gov.eg/course/view.php?id=2052>



# Agenda

- A. The History and Evolution of Java
- B. An Overview of Java
- C. Data Types, Variables and Arrays
- D. Operators, Control Statements and String Handling
- E. Modifiers, Access Specifiers, Packages and Interfaces
- F. Wrapper Classes, Autoboxing and Inner Classes
- G. Exception Handling
- H. Generics and Lambda Expressions and Method Reference
- I. Java Stream API
- J. Java Collections
- K. Multi-Threading in Java

# Agenda

- A. The History and Evolution of Java
- B. An Overview of Java
- C. Data Types, Variables and Arrays**
- D. Operators, Control Statements and String Handling
- E. Modifiers, Access Specifiers, Packages and Interfaces
- F. Wrapper Classes, Autoboxing and Inner Classes
- G. Exception Handling
- H. Generics and Lambda Expressions and Method Reference
- I. Java Stream API
- J. Java Collections
- K. Multi-Threading in Java

# Lesson 3

Data Types, Variables, and Arrays



# Introduction

- ▶ As with all modern programming languages, Java supports several types of datatypes. You may use these datatypes to declare variables and to create arrays.
- ▶ ***Java Is a Strongly Typed Language***
  - ▶ Every variable has a type, every expression has a type, and every type is strictly defined.
  - ▶ All assignments, whether explicit or via parameter passing in method calls, are checked for type compatibility.
  - ▶ The Java compiler checks all expressions and parameters to ensure that the types are compatible.

# Data types

- ▶ Data types can be classified into two types:

## Primitive

|                |              |                                                                  |
|----------------|--------------|------------------------------------------------------------------|
| <b>boolean</b> | <b>1 bit</b> | <b>(true/false)</b>                                              |
| <b>byte</b>    | <b>1 B</b>   | <b>(-2<sup>7</sup> → 2<sup>7</sup>-1) (-128 → +127)</b>          |
| <b>short</b>   | <b>2 B</b>   | <b>(-2<sup>15</sup> → 2<sup>15</sup>-1) (-32,768 to +32,767)</b> |
| <b>int</b>     | <b>4 B</b>   | <b>(-2<sup>31</sup> → 2<sup>31</sup>-1)</b>                      |
| <b>long</b>    | <b>8 B</b>   | <b>(-2<sup>63</sup> → 2<sup>63</sup>-1)</b>                      |
| <b>float</b>   | <b>4 B</b>   | <b><u>Standard</u>: IEEE 754 Specification</b>                   |
| <b>double</b>  | <b>8 B</b>   | <b><u>Standard</u>: IEEE 754 Specification</b>                   |
| <b>char</b>    | <b>2 B</b>   | <b>unsigned Unicode chars (0 → 2<sup>16</sup>-1)</b>             |

## Reference

**Arrays**

**Classes**

**Interfaces**

# The Primitive Types

- ▶ Java defines eight primitive types of data, that can be put in four groups:
- ▶ **Integers:** This group includes **byte**, **short**, **int**, and **long**, which are for whole-valued signed numbers.
- ▶ **Floating-point numbers:** This group includes **float** and **double**, which represent numbers with fractional precision.
- ▶ **Characters:** This group includes **char**, which represents symbols in a character set, like letters and numbers.
- ▶ **Boolean:** This group includes **boolean**, which is a special type for representing true/false values.



# The Primitive Types

- ▶ The primitive types represent single values—not complex objects. Although Java is otherwise completely object-oriented, the primitive types are not.
- ▶ The reason for this is efficiency. Making the primitive types into objects would have degraded performance too much.
- ▶ The primitive types are defined to have an explicit range and mathematical behavior.
- ▶ Languages such as C and C++ allow the size of an integer to vary based upon the dictates of the execution environment.

```
int x = 5;
```

5

# Integer Types

| Name  | Width (in bits) | Range                                                      |
|-------|-----------------|------------------------------------------------------------|
| long  | 64              | -9,223,372,036,854,775,808 to<br>9,223,372,036,854,775,807 |
| int   | 32              | -2,147,483,648 to 2,147,483,647                            |
| short | 16              | -32,768 to 32,767                                          |
| byte  | 8               | -128 to 127                                                |

**variable declaration examples:**

byte b , c;

short s;

short t;

# Floating-Point Types

- ▶ Floating-point numbers, also known as real numbers, are used when evaluating expressions that require fractional precision.
- ▶ Java implements the standard (IEEE-754) set of floating-point types and operators.
- ▶ There are two kinds of floating-point types, float and double their width and ranges are shown here:

| Name   | Width in Bits | Approximate Range      |
|--------|---------------|------------------------|
| double | 64            | 4.9 e-324 to 1.8 e+308 |
| float  | 32            | 1.4 e-45 to 3.4 e+38   |

# Characters

- ▶ In Java, the data type used to store characters is **char**.
- ▶ Java uses Unicode to represent characters.
- ▶ Unicode defines a fully international character set that can represent all of the characters found in all human languages.
- ▶ At the time of Java's creation, Unicode required 16 bits. Thus, in Java char is a 16-bit type.
- ▶ The standard set of characters known as ASCII still ranges from 0 to 127 as always, and the extended 8-bit character set, ISO-Latin-1, ranges from 0 to 255.

# Booleans

- ▶ Java has a primitive type, called boolean, for logical values.
- ▶ It can have only one of two possible values, **true** or **false**.
- ▶ This is the type returned by all relational operators, as in the case of **a < b**. **boolean** is also the type required by the conditional expressions that govern the control statements such as **if** and **for**.

# Literals

- ▶ literals are the values that could be assigned to primitive data types or strings.
- ▶ literals are categorized according to the data types categories, **Integer, Floating-Point, Boolean, and Character Literals**

# Literals

- ▶ A literal is any value that can be assigned to a primitive data type or String.

|                          |                              |                  |
|--------------------------|------------------------------|------------------|
| boolean                  | true                         | false            |
| char                     | 'a' .... 'z'    'A' .... 'Z' |                  |
|                          | '\\u0000' .... '\\uFFFF'     |                  |
|                          | '\\n'   '\\r'   '\\t'        |                  |
| Integral data type       | 15                           | Decimal (int)    |
|                          | 15L                          | Decimal (long)   |
|                          | 017                          | Octal            |
|                          | 0XF                          | Hexadecimal      |
| Floating point data type | 73.8                         | double           |
|                          | 73.8F                        | float            |
|                          | 5.4 E-70                     | $5.4 * 10^{-70}$ |
|                          | 5.4 e+70                     | $5.4 * 10^{70}$  |

# Integer Literals

```
int x=10;  
byte b=15;  
short s= 23;  
long l=15;
```

```
int x=012;  
byte b=0x15;  
short s= 0X23;  
long l=15L;
```



# Floating-Point Literals

- ▶ Floating-point numbers represent decimal values with a fractional component.
- ▶ They can be expressed in either *standard* or *scientific* notation.

## ***Standard Notation***

double d=3.14159;

double d1=0.6667;

## ***Scientific Notation***

double d=314159E-05;

double d1=6667e-4;

- ▶ Floating-point literals in Java default to double precision.
- ▶ To specify a float literal, you must append an F or f to the constant.

# Boolean Literals

- ▶ There are only two logical values that a boolean value can have, ***true*** and ***false***.
- ▶ ***The values of true and false do not convert into any numerical representation.***
- ▶ ***The true literal in Java does not equal 1, nor does the false literal equal 0.***

# Character Literals

- ▶ A literal character is represented inside a pair of single quotes.
- ▶ All of the visible ASCII characters can be directly entered inside the quotes, such as 'a', 'z', and '@'.
- ▶ For characters that are impossible to enter directly, there are several escape sequences that allow you to enter the character you need, such as ' \' ' for the single-quote character itself and ' \n ' for the newline character.

# Character Literals

- ▶ For octal notation, use the backslash followed by the three digit number. For example, `'\141'` is the letter 'a'. For hexadecimal, you enter a backslash-u ( `\u` ), then exactly four hexadecimal digits.
- ▶ For example, `'\u0061'` is the ISO-Latin-1 'a' because the top byte is zero.

| Escape Sequence   | Description     | Escape Sequence     | Description                   |
|-------------------|-----------------|---------------------|-------------------------------|
| <code>\ddd</code> | Octal character | <code>\uxxxx</code> | Hexadecimal Unicode character |
| <code>\'</code>   | Single quote    | <code>\"</code>     | Double quote                  |
| <code>\\</code>   | Backslash       | <code>\r</code>     | Carriage return               |
| <code>\n</code>   | New Line        | <code>\f</code>     | Form feed                     |
| <code>\t</code>   | Tab             | <code>\b</code>     | Backspace                     |

# Variables

- ▶ The variable is the basic unit of storage in a Java program.
- ▶ A variable is defined by the combination of:
  - ▶ an identifier,
  - ▶ a type,
  - ▶ and an optional initializer.
- ▶ In addition, all variables have a scope, which defines their **visibility**, and a lifetime.

# Variables

## ▶ Declaring a Variable

- ▶ All variables must be declared before they can be used.
- ▶ The basic form of a variable declaration is shown here:

***type identifier [ = value ][, identifier [= value ] ...];***

***int a, b, c;***

***int d= 3, e, f=5;***

***byte z= 22;***

***double pi=3.14159;***

***char x='x';***

# Variables

## ► Dynamic Initialization

- The preceding examples have used only constants as initializers
- Java allows variables to be initialized dynamically, using any expression valid at the time the variable is declared.

```
double a=3.0, b=4.0;
```

```
double c= Math.sqrt (a*a + b*b);
```

# Variables

## ▶ The Scope and Lifetime of Variables

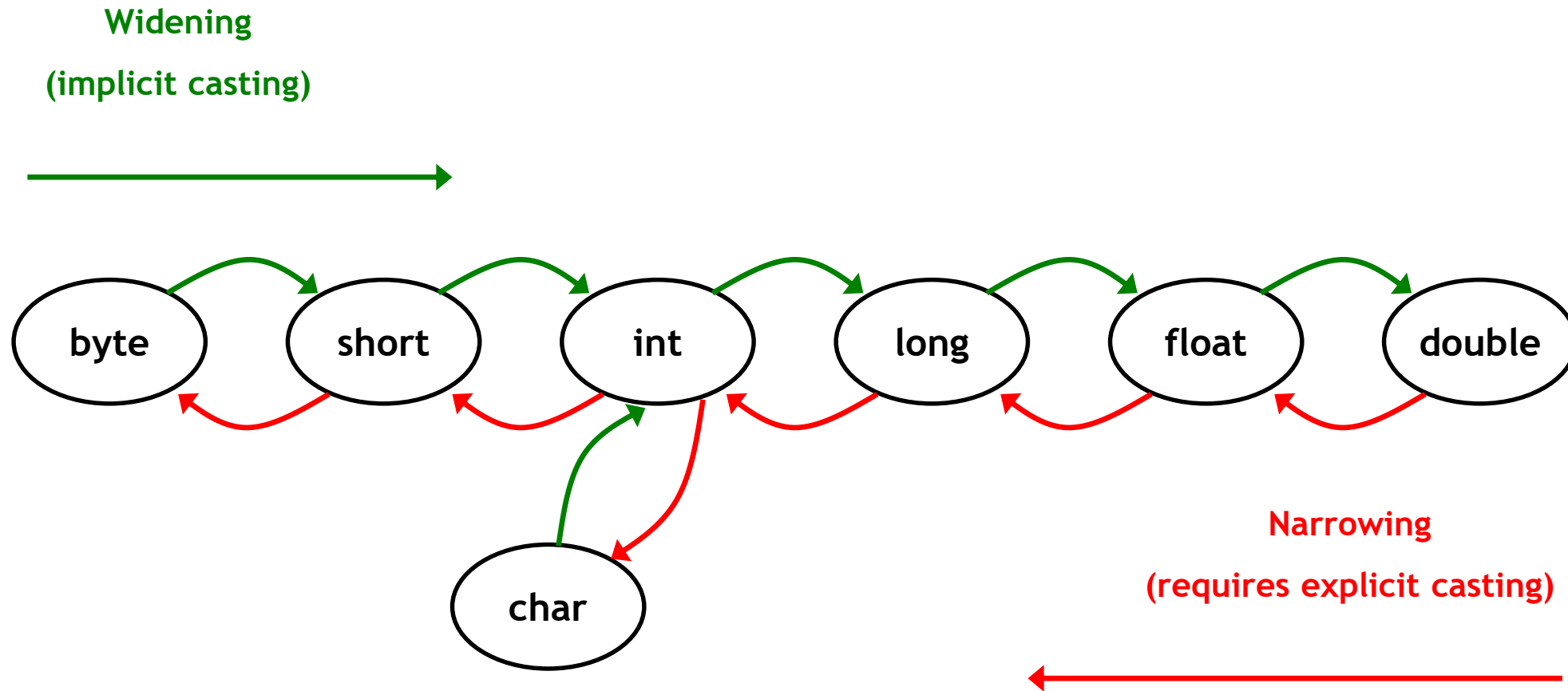
- ▶ Java allows variables to be declared within any block
- ▶ A block defines a scope. Thus, each time you start a new block, you are creating a new scope.
- ▶ A scope determines what objects are visible to other parts of your program. It also determines the lifetime of those objects.
- ▶ The two major scopes are those defined by a class and those defined by a method.
- ▶ The scope defined by a method begins with its opening curly brace. However, if that method has parameters, they too are included within the method's scope.



# Variables- Scope Example

```
//Demonstrate block scope.  
class Scope {  
  public static void main(String args[]) {  
    int x; // known to all code within main  
    x = 10;  
    if(x == 10) { // start new scope  
      int y = 20; // known only to this block  
      // x and y both known here.  
      System.out.println("x and y: " + x + " " + y);  
      x = y * 2;  
    } // y = 100; // Error! y not known here // x is still known here.  
    System.out.println("x is " + x);  
  }  
}
```

# Type Conversion and Casting



# Type Conversion and Casting

- ▶ It is fairly common to assign a value of one type to a variable of another type. If the two types are compatible, then Java will perform the conversion automatically.
- ▶ When one type of data is assigned to another type of variable, an automatic type conversion will take place if the following two conditions are met:
  - ▶ *The two types are compatible.*
  - ▶ *The destination type is larger than the source type.*
- ▶ When these two conditions are met, a **widening conversion** takes place.

# Type Conversion and Casting

- ▶ For widening conversions, the numeric types, including integer and floating-point types, are compatible with each other.
- ▶ However, there are no automatic conversions from the numeric types to *char* or *boolean*. Also, *char* and *boolean* are not compatible with each other.
- ▶ Java also performs an automatic type conversion when storing a literal integer constant into variables of type *byte*, *short*, *long*, or *char*

# Casting Incompatible Types

- ▶ What if you want to assign an `int` value to a `byte` variable?
- ▶ This conversion will not be performed automatically, because a *byte* is smaller than an *int*.
- ▶ This kind of conversion is sometimes called a narrowing conversion, since you are explicitly making the value narrower so that it will fit into the target type.
- ▶ To create a conversion between two incompatible types, you must use a cast. A cast is simply an explicit type conversion. It has this general form: ***(target-type) value***

```
int a;  
byte b;  
b=(byte) a
```

# Automatic Type Promotion in Expressions

- ▶ In addition to assignments, there is another place where certain type conversions may occur: in expressions.
- ▶ In an expression, the precision required of an intermediate value will sometimes exceed the range of either operand.  
For example, examine the following expression:

***byte a=40;***

***byte b=50;***

***byte c =100;***

***int d=a \* b / c;***

# Data types

- ▶ Data types can be classified into two types:

## Primitive

|                |              |                                                                                   |
|----------------|--------------|-----------------------------------------------------------------------------------|
| <b>boolean</b> | <b>1 bit</b> | <b>(true/false)</b>                                                               |
| <b>byte</b>    | <b>1 B</b>   | <b>(<math>-2^7 \rightarrow 2^7-1</math>) (-128 <math>\rightarrow</math> +127)</b> |
| <b>short</b>   | <b>2 B</b>   | <b>(<math>-2^{15} \rightarrow 2^{15}-1</math>) (-32,768 to +32,767)</b>           |
| <b>int</b>     | <b>4 B</b>   | <b>(<math>-2^{31} \rightarrow 2^{31}-1</math>)</b>                                |
| <b>long</b>    | <b>8 B</b>   | <b>(<math>-2^{63} \rightarrow 2^{63}-1</math>)</b>                                |
| <b>float</b>   | <b>4 B</b>   | <b><u>Standard</u>: IEEE 754 Specification</b>                                    |
| <b>double</b>  | <b>8 B</b>   | <b><u>Standard</u>: IEEE 754 Specification</b>                                    |
| <b>char</b>    | <b>2 B</b>   | <b>unsigned Unicode chars (0 <math>\rightarrow</math> <math>2^{16}-1</math>)</b>  |

## Reference

**Arrays**

**Classes**

**Interfaces**

# Reference Data types: Classes

- ▶ General syntax for creating an object:

```
MyClass myRef;           // just a reference
```

```
myRef = new MyClass();   // construct a new object
```

- ▶ Or on one line:

```
MyClass myRef = new MyClass();
```

- ▶ An object is garbage collected when there is no reference pointing to it.



# Reference Data types: Classes

```

Test str1;           // just a null reference
str1 = new Test("Hello"); // object construction

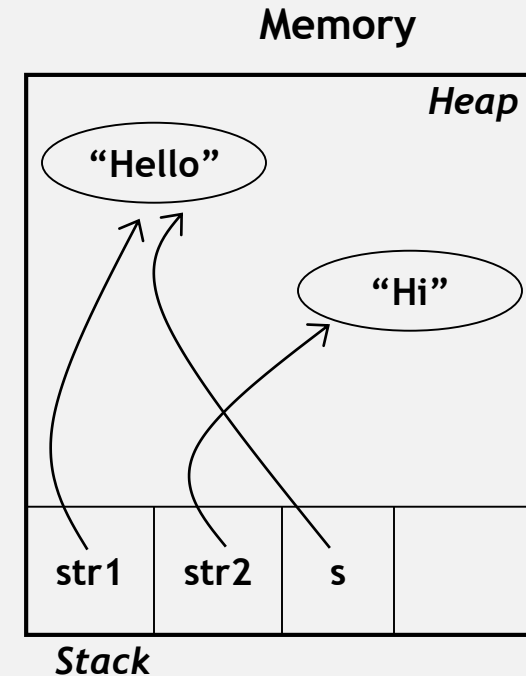
Test str2 = new Test("Hi");

Test s = str1;       //two references to the same object

str1 = null;

s = null; // The object containing "Hello" will
           // now be eligible for garbage collection.

str1.anyMethod(); // ILLEGAL!
                 //Throws NullPointerException
    
```



# Arrays

- ▶ An array is a group of like-typed variables that are referred to by a common name.
- ▶ Arrays of any type can be created and may have one or more dimensions.
- ▶ A specific element in an array is accessed by its index.
- ▶ Arrays offer a convenient means of grouping related information.

# Declaring an Array

## ► Example1: Array of Primitives:

```
int[] myArr;
```

```
myArr = new int[3];
```

```
myArr[0] = 15 ;
```

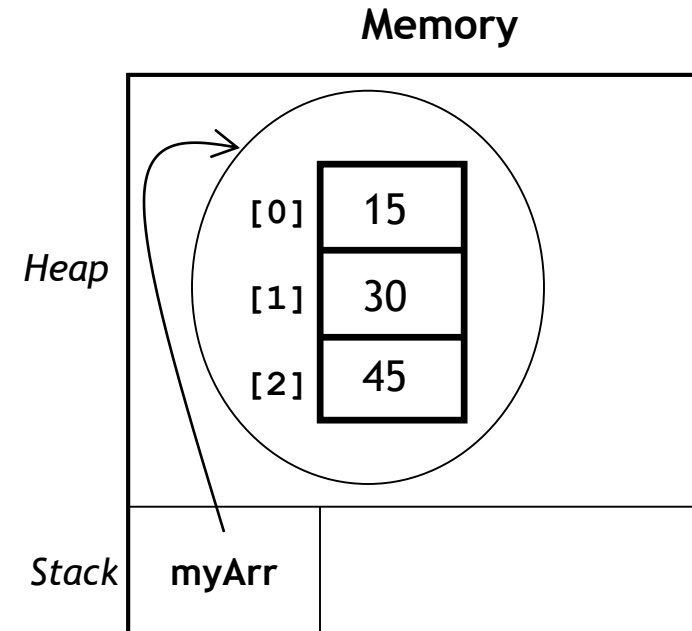
```
myArr[1] = 30 ;
```

```
myArr[2] = 45 ;
```

```
System.out.println(myArr[2]) ;
```

```
myArr[3] = ... ; // ILLEGAL!
```

```
//Throws ArrayIndexOutOfBoundsException
```



# One-Dimensional Arrays

- ▶ A one-dimensional array is, essentially, a list of like-typed variables.
- ▶ To create an array, you first must create an array variable of the desired type.
- ▶ The general form of a one-dimensional array declaration is

*type var\_name[ ];*

- ▶ To link *month\_days* with an actual, physical array of integers, you must allocate one using *new* and assign it to *month\_days*.
- ▶ *new* is a special operator that allocates memory.

 **int month\_days [ ];**

# One-Dimensional Arrays

- ▶ The general form of *new* as it applies to one dimensional arrays appears as follows:

***array-var = new type [size];***



***month\_days=new int [12];***

- ▶ After this statement executes, **month\_days** will refer to an array of 12 integers.
- ▶ Further, all elements in the array will be initialized to zero (The default value of the underlying data type).

# One Dimensional Array Example

//Demonstrate a one-dimensional array.

```
class ArrayTest {  
    public static void main(String args[]) {  
        int month_days[];  
        month_days = new int[12];  
        month_days[0] = 31; month_days[1] = 28;  
        month_days[2] = 31; month_days[3] = 30;  
        month_days[4] = 31; month_days[5] = 30;  
        month_days[6] = 31; month_days[7] = 31;  
        month_days[8] = 30; month_days[9] = 31;  
        month_days[10] = 30; month_days[11] = 31;  
        System.out.println("April has " + month_days[3] + " days.");  
    }  
}
```

# Declaring an Array

## ► Example2: Array of Object References:

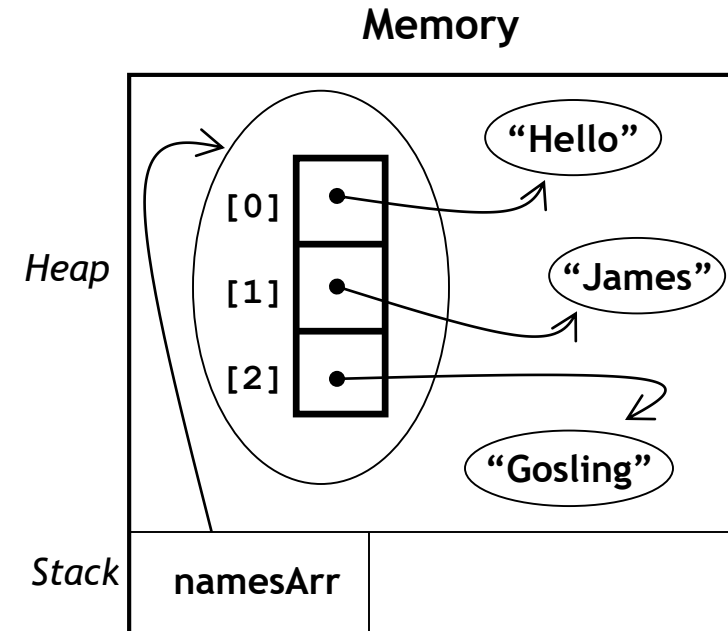
```
Test[] namesArr;
```

```
namesArr = new Test[3];
```

```
namesArr[0].anyMethod() // ILLEGAL!  
                        //Throws NullPointerException
```

```
namesArr[0] = new Test("Hello");  
namesArr[1] = new Test("James");  
namesArr[2] = new Test("Gosling");
```

```
System.out.println(namesArr[1]);
```



# Multidimensional Arrays

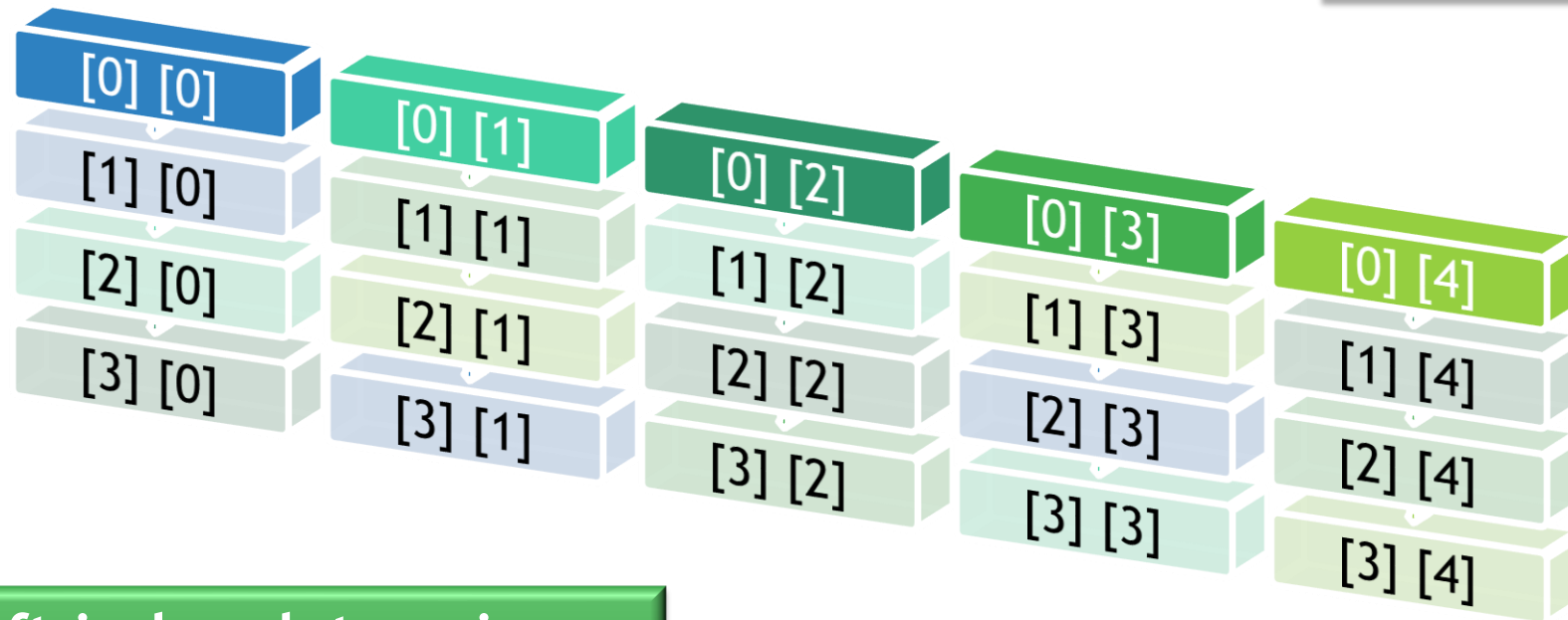
- ▶ *Multidimensional* arrays are implemented as arrays of arrays.
- ▶ To declare a multidimensional array variable, specify each additional index using another set of square brackets.

```
int twoD [ ] [ ] = new int [ 4 ] [ 5 ]
```

- ▶ This allocates a 4 by 5 array and assigns it to twoD. Internally, this matrix is implemented as an array of arrays of int.
- ▶ Conceptually, this array will look like the one shown in the following figure



# 2-Dimensional Array Conceptual View



Right index determines  
column

Left index determines  
row

# Agenda

- A. The History and Evolution of Java
- B. An Overview of Java
- C. Data Types, Variables and Arrays
- D. Operators, Control Statements and String Handling**
- E. Modifiers, Access Specifiers, Packages and Interfaces
- F. Wrapper Classes, Autoboxing and Inner Classes
- G. Exception Handling
- H. Generics and Lambda Expressions and Method Reference
- I. Java Stream API
- J. Java Collections
- K. Multi-Threading in Java

# Lesson 4

Operators, Control Statements, and String Handling



# Operators

- ▶ Arithmetic Operators
- ▶ Bitwise Operators
- ▶ Relational Operators
- ▶ Boolean Logical Operators
- ▶ The Ternary Operator (?)
- ▶ **instanceof** Operator
- ▶ The arrow Operator (->)

# Arithmetic Operators

| Operator | Result                         |
|----------|--------------------------------|
| +        | Addition (also unary plus)     |
| -        | Subtraction (also unary minus) |
| *        | Multiplication                 |
| /        | Division                       |
| %        | Modulus                        |
| ++       | Increment                      |
| +=       | Addition Assignment            |
| -=       | Subtraction Assignment         |
| *=       | Multiplication assignment      |
| /=       | Division Assignment            |
| %=       | Modulus Assignment             |
| --       | Decrement                      |

- The operands of the arithmetic operators must be of a numeric type.
- Remember that when the division operator is applied to an integer type, there will be no fractional component attached to the result.

# The Bitwise Operators

- ▶ Java defines several bitwise operators that can be applied to the integer types: **long**, **int**, **short**, **char**, and **byte**.
- ▶ These operators act upon the individual bits of their operands. They are summarized in the following table:

# The Bitwise Operators

| Operator | Result                | Operator | Result                           |
|----------|-----------------------|----------|----------------------------------|
| ~        | Bitwise unary Not     | &=       | Bitwise AND Assignment           |
| &        | Bitwise AND           | =        | Bitwise OR Assignment            |
|          | Bitwise OR            | ^=       | Bitwise Exclusive OR Assignment  |
| ^        | Bitwise Exclusive OR  | >>=      | Shift Right Assignment           |
| >>       | Shift Right           | >>>=     | Shift Right zero fill Assignment |
| >>>      | Shift Right zero fill | <<=      | Shift left Assignment            |
| <<       | Shift left            |          |                                  |

# The Bitwise Operators

- Assume if  $a = 60$ ; and  $b = 13$ ;

$a = 0011\ 1100$

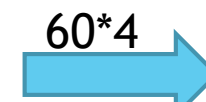
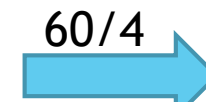
$b = 0000\ 1101$

$$= 0 \cdot 2^0 + 0 \cdot 2^1 + 1 \cdot 2^2 + 1 \cdot 2^3 + 1 \cdot 2^4 + 1 \cdot 2^5 + 0 \cdot 2^6 + 0 \cdot 2^7$$

$a \& b = 0000\ 1100$

$a | b = 0011\ 1101$

$a \wedge b = 0011\ 0001$

|            |                                                                                      |          |                                                                                       |     |
|------------|--------------------------------------------------------------------------------------|----------|---------------------------------------------------------------------------------------|-----|
| $a \ll 2$  |    | 11110000 |    | 240 |
| $a \gg 2$  |    | 00001111 |    | 15  |
| $a \ggg 2$ |  | 00001111 |  | 15  |
| $\sim a$   |  | 11000011 |  | -61 |



# Relational Operators

- ▶ The relational operators determine the relationship that one operand has to the other
- ▶ They determine equality and ordering.
- ▶ The relational operators are shown here:

| Operator | Result       | Operator | Result                   |
|----------|--------------|----------|--------------------------|
| ==       | Equal to     | <        | Less than                |
| !=       | Not equal to | >=       | Greater than or equal to |
| >        | Greater than | <=       | Less than or equal to    |

- ▶ The outcome of these operations is a boolean value.
- ▶ The relational operators are most frequently used in the expressions that control the if statement and the various loop statements.

# Boolean Logical Operators

- ▶ The Boolean logical operators shown below, operate only on boolean operands.
- ▶ All of the binary logical operators combine two boolean values to form a resultant boolean value.

| Operator | Result                     | Operator | Result            |
|----------|----------------------------|----------|-------------------|
| &        | Logical AND                | &&       | Short-circuit AND |
|          | Logical OR                 |          | Short-circuit OR  |
| ^        | Logical XOR (exclusive OR) | &=       | AND assignment    |
| !        | Logical Unary Not          | =        | OR assignment     |
| ==       | Equal to                   | !=       | Not equal to      |

# Boolean Logical Operators

| A     |  |  |  |  | B     |
|-------|--|--|--|--|-------|
| False |  |  |  |  | False |
| True  |  |  |  |  | False |
| False |  |  |  |  | True  |
| True  |  |  |  |  | True  |

# Boolean Operators Example

```
//Demonstrate the boolean logical operators.  
  
class BoolLogic {  
    public static void main(String args[]) {  
        boolean a = true;  
        boolean b = false;  
        boolean c = a | b;  
        boolean d = a & b;  
        boolean e = a ^ b;  
        boolean f = (!a & b) | (a & !b);  
        boolean g = !a;  
        System.out.println("a = " + a); System.out.println("b = " + b);  
        System.out.println("a | b = " + c); System.out.println("a & b = " + d);  
        System.out.println("a ^ b = " + e); System.out.println("!a & b | a & !b = " + f);  
        System.out.println("!a = " + g);  
    }  
}
```

# The Ternary Operator (?)

- ▶ Java includes a special ternary (three-way) operator that can replace certain types of if-then-else statements.
- ▶ The ? has this general form:

***expression1 ? expression2 : expression3***

- ▶ Here, expression1 can be any expression that evaluates to a boolean value. If expression1 is true, then expression2 is evaluated; otherwise, expression3 is evaluated.

***double ratio= total == 0 ? 0 : num / total***

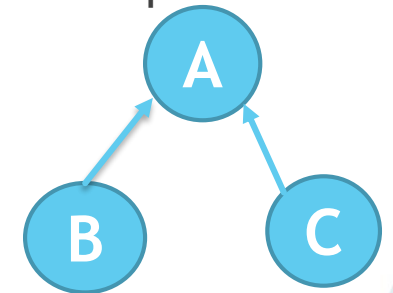
# instanceof Operator

- ▶ knowing the type of an object during runtime is useful.
- ▶ For example, you might have one thread of execution that generates various types of objects, and another thread that processes these objects.
- ▶ In this situation, it might be useful for the processing thread to know the type of each object when it receives it.
- ▶ Another situation in which knowledge of an object's type at run time is important involves casting. In Java, an invalid cast causes a run-time error.

# instanceof Operator

- ▶ For example, a superclass called A can produce two subclasses, called B and C. Thus, casting a B object into type A or casting a C object into type A is legal, but casting a B object into type C (or vice versa) isn't legal.
- ▶ Java provides the run-time operator instanceof to answer this question. The instanceof operator has this general form:

**objref instanceof type**



- ▶ Here, objref is a reference to an instance of a class, and type is a class type.

# instanceof example

```
class A {  
    int i, j;  
}  
class B {  
    int i, j;  
}  
class C extends A {  
    int k;  
}  
class D extends A {  
    int k;  
}
```

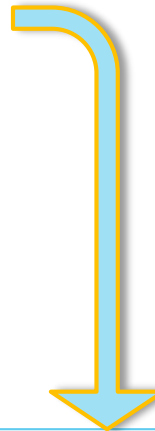


# The arrow Operator (->)

- ▶ As of JDK 8, a new feature has been added to Java that profoundly enhances the expressive power of the language.
- ▶ This feature is the *lambda expression*.
- ▶ Not only do lambda expressions add new syntax elements to the language, they also streamline the way that certain common constructs are implemented.
- ▶ The addition of lambda expressions have also provided the catalyst for other new Java features (*Default Methods-Method Reference*).

# Lambda Expression

- A *lambda expression* is, essentially, an anonymous (that is, unnamed) method.
- However, this method is not executed on its own. Instead, it is used to implement a method defined by a functional interface.
- Thus, a lambda expression results in a form of anonymous class.
- Lambda expressions are also commonly referred to as *closures*.



*A functional interface* is an interface that contains one and only one abstract method

# Lambda Expression

- ▶ The lambda expression introduces a new syntax element and operator into the Java language. The new operator, sometimes referred to as the *lambda operator* or the *arrow operator*, is **->**.
- ▶ It divides a lambda expression into two parts.
- ▶ The left side specifies any parameters required by the lambda expression. On the right side is the *lambda body*, which specifies the actions of the lambda expression.

# Single Lambda Expressions(Examples)

`() -> 98.6`



- It evaluates to a constant value.
- This lambda expression takes no parameters, thus the parameter list is empty.
- It returns the constant value 98.6.
- Therefore, it is similar to the following method:
  - `double myMeth() { return 98.6; }`
- Of course the method defined by a lambda expression does not have a name

`() -> Math.random()*100`



This lambda expression obtains a pseudo-random value from **Math.random()**, multiplies it by 100, and returns the result. It, too, does not require a parameter.

# Single Lambda Expressions(Examples)

- When a lambda expression requires a parameter, it is specified in the parameter list on the left side of the lambda operator. Here is a simple example:

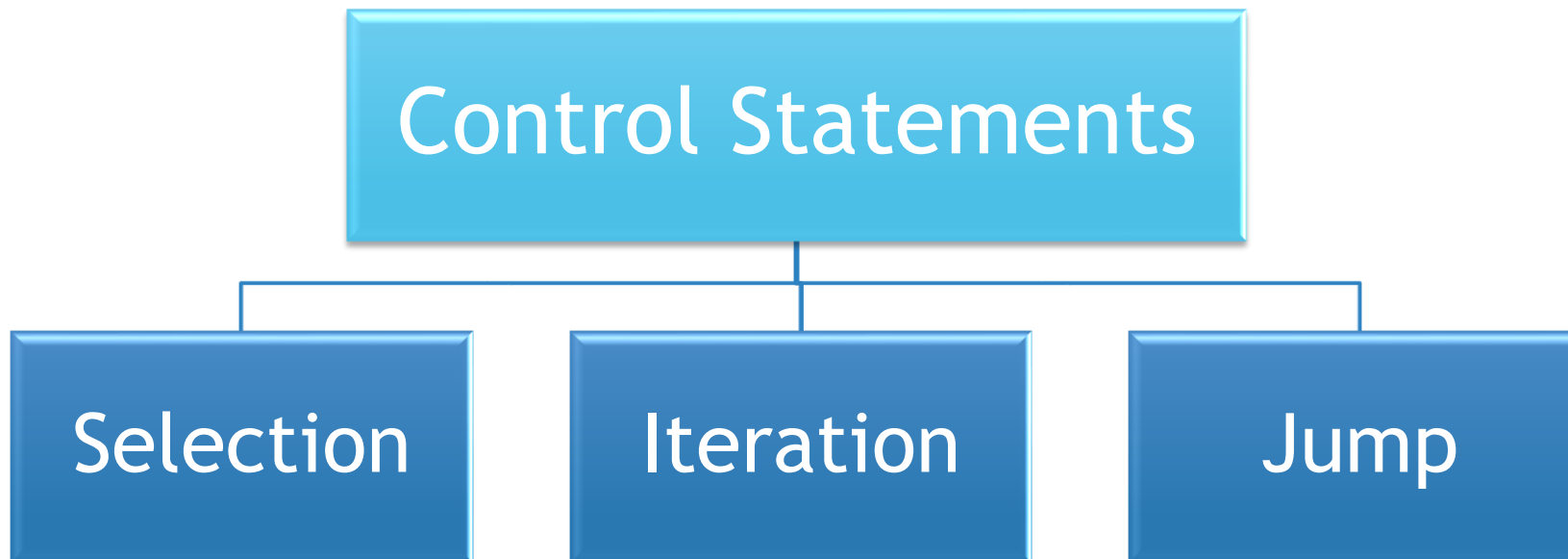
**$(n) \rightarrow 1.0 / n$**

- This lambda expression returns the reciprocal of the value of  $n$
- If  $n$  is 4.0 then the reciprocal is 0.25
- The type of  $n$  can be inferred so you don't need to explicitly specify it.
- Like a named method, a lambda expression can specify as many parameters as needed

**$(n) \rightarrow (n \% 2) == 0$**

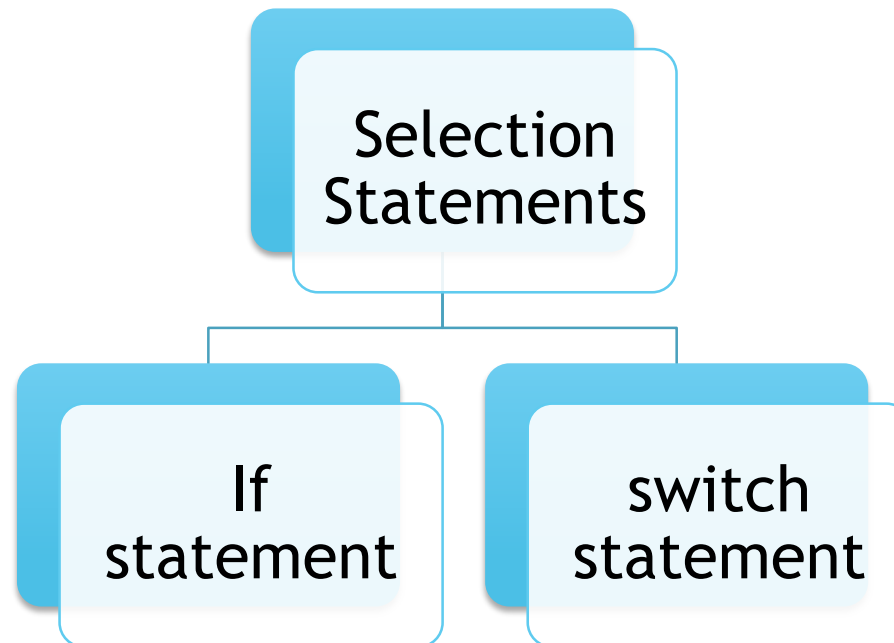
- Any valid type can be used as the return type of a lambda expression.
- The above lambda expression return ***boolean***
- When a lambda expression has only one parameter, it is not necessary to surround the parameter name with parentheses when it is specified on the left side.

# Control Statements in Java

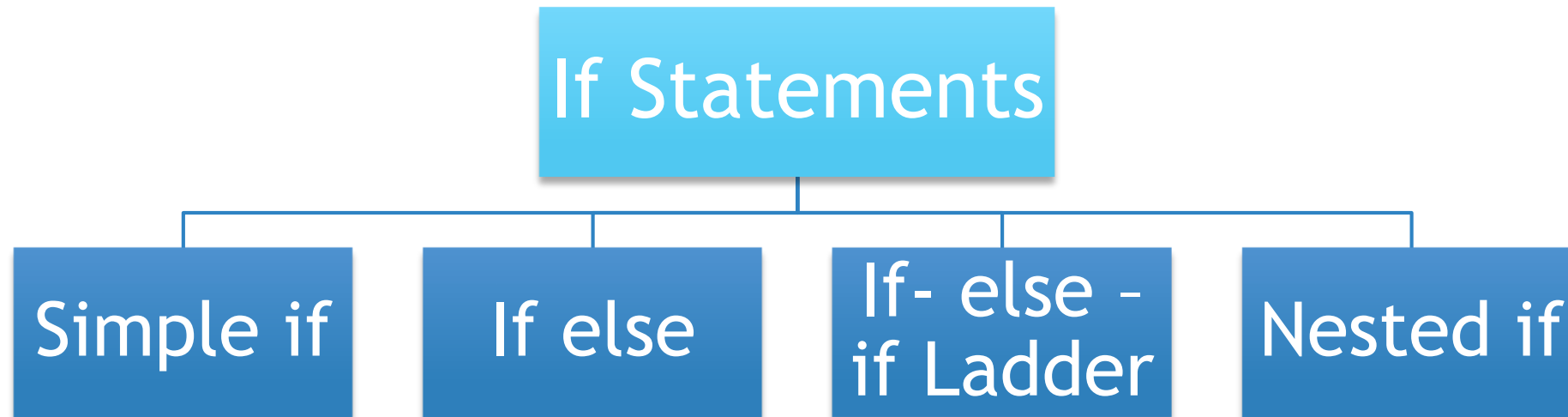


# Selection Statements

- Selection Statements are also called Decision Making Statements.



# If Statements





# Simple if

## Syntax :

```
if (condition)
{
    statement1;
}
```

```
if(x < y) {
    System.out.println("x is less than y");
}
```

**Purpose:** The statements will be evaluated if the value of the condition is true.

# If else

## Syntax :

```
if (condition)
{
    statement1;
}
else
{
    statement2;
}
```

```
if(x < y) {
    System.out.println("x is less than y");
}else {
    System.out.println("x is either equals y or greater
than y");
}
```

**Purpose:** The statement 1 is evaluated if the value of the condition is true otherwise statement 2 is true.

# If-else-if Ladder

**Syntax :**

```
if(condition)
    statements;
else if(condition)
    statements;
    else if(condition)
        statements;
    ...
    ...
else
    statements;
```

# If-else-if example

```
class IfElse {  
    public static void main(String args[]) {  
        int month = 4; // April  
        String season;  
  
        if(month == 12 || month == 1 || month == 2)  
            season = "Winter";  
        else if(month == 3 || month == 4 || month == 5)  
            season = "Spring";  
        else if(month == 6 || month == 7 || month == 8)  
            season = "Summer";  
        else if(month == 9 || month == 10 || month == 11)  
            season = "Autumn";  
        else  
            season = "Unknown Month";  
        System.out.println("April is in the " + season + ".");  
    }  
}
```

# Nested if

- ▶ A nested if is an if statement that is the target of another if or else.
- ▶ Nested ifs are very common in programming.

**Syntax :**

```
if(condition){  
    if(condition) statements....  
    else  
    statements....  
}else{  
    if(condition)  
    statements....  
    else  
    statements....  
}
```

# Example

```
class NestedIfDemo
{
    public static void main(String args[])
    {
        int i = 10;

        if (i == 10)
        {
            // First if statement
            if (i < 15)
                System.out.println("i is smaller than 15");
            // Nested - if statement
            // Will only be executed if statement above
            // it is true
            if (i < 12)
                System.out.println("i is smaller than 12 too");
            else
                System.out.println("i is greater than 15");
        }
    }
}
```

# switch case

- ▶ **Purpose:** The statements N will be evaluated if the value of the expression matches the value N.

```
Syntax :  
switch (expression)  
{  
  case value 1 :  
    statement 1 ;  
    break;  
  case value 2 :  
    statement 2 ;  
    break;  
  ...  
  case value N : statement N ; break;  
  default :  
    statements ; break;  
}
```

# Example

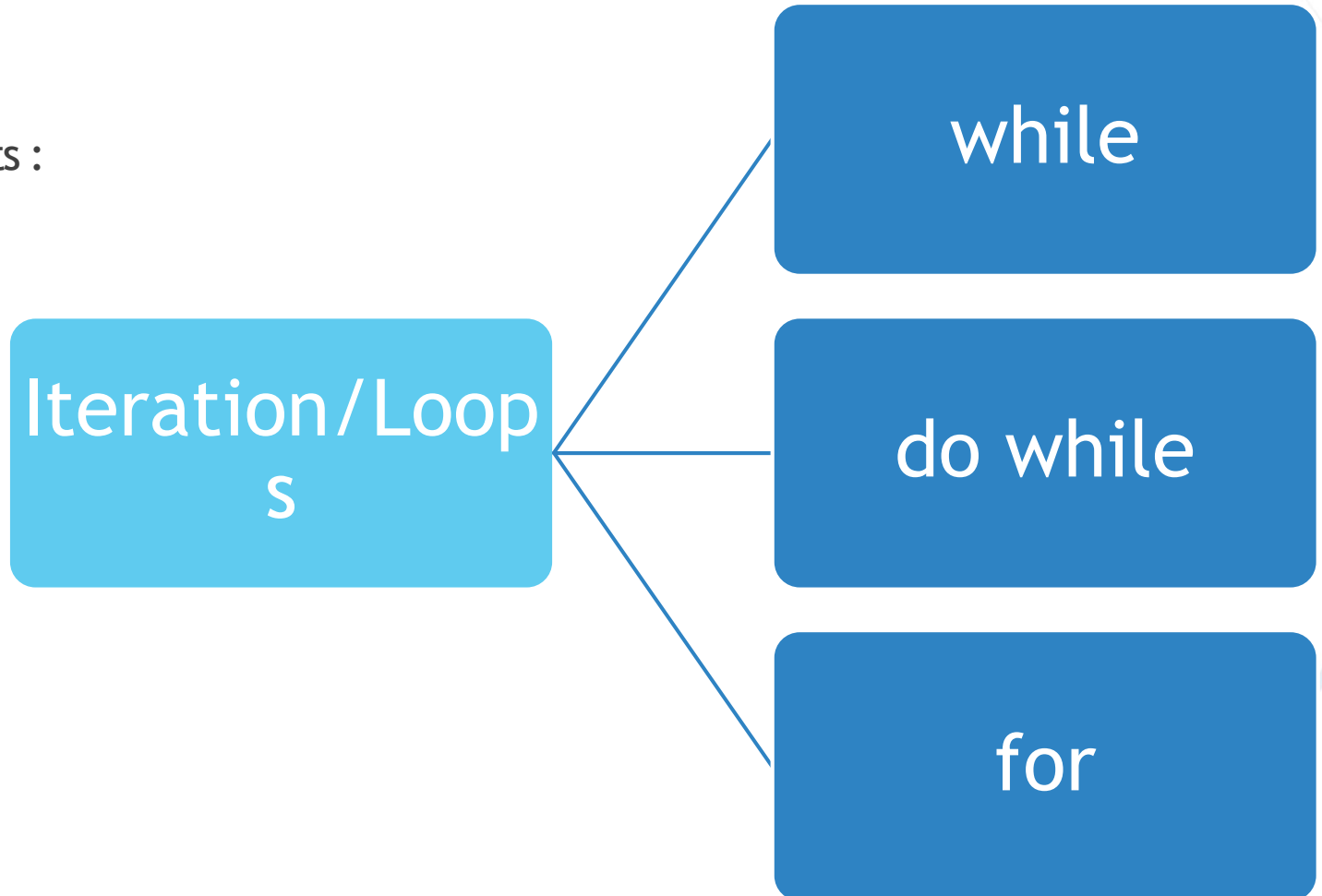
```
class SwitchCaseDemo
{
    public static void main(String args[])
    {
        int i = 9;
        switch (i)
        {
            case 0:
                System.out.println("i is zero.");
                break;
            case 1:
                System.out.println("i is one.");
                break;
            case 2:
                System.out.println("i is two.");
                break;
            default:
                System.out.println("i is greater than 2.");
        }
    }
}
```



# Iteration Statements

► Each loop has four types of statements :

- *Initialization*
- *Condition checking*
- *Execution*
- *Increment / Decrement*



# while loop

## ▶ Syntax

```
initialization  
while(final value)  
{  
    statements;  
    increment/decrement;  
}
```

- ▶ **Purpose:** To evaluate the statements from initial value to final value with given increment/decrement.

```
int m=1;  
while(m<=20){  
    System.out.println(m);  
    m=m+1;  
}
```

# do while loop

- ▶ Syntax

**initialization**

**do{**

**statements;**

**increment/decrement;**

**}**

**while(final value);**

- ▶ **Purpose:** To evaluate the statements from initial value to final value with given increment/decrement.

```
int m=1;  
do{  
    System.out.println(m);  
    m=m+1;  
} while(m<=20);
```

# for loop

- ▶ **Syntax**

**for(initialization; termination condition; step)**

**{**

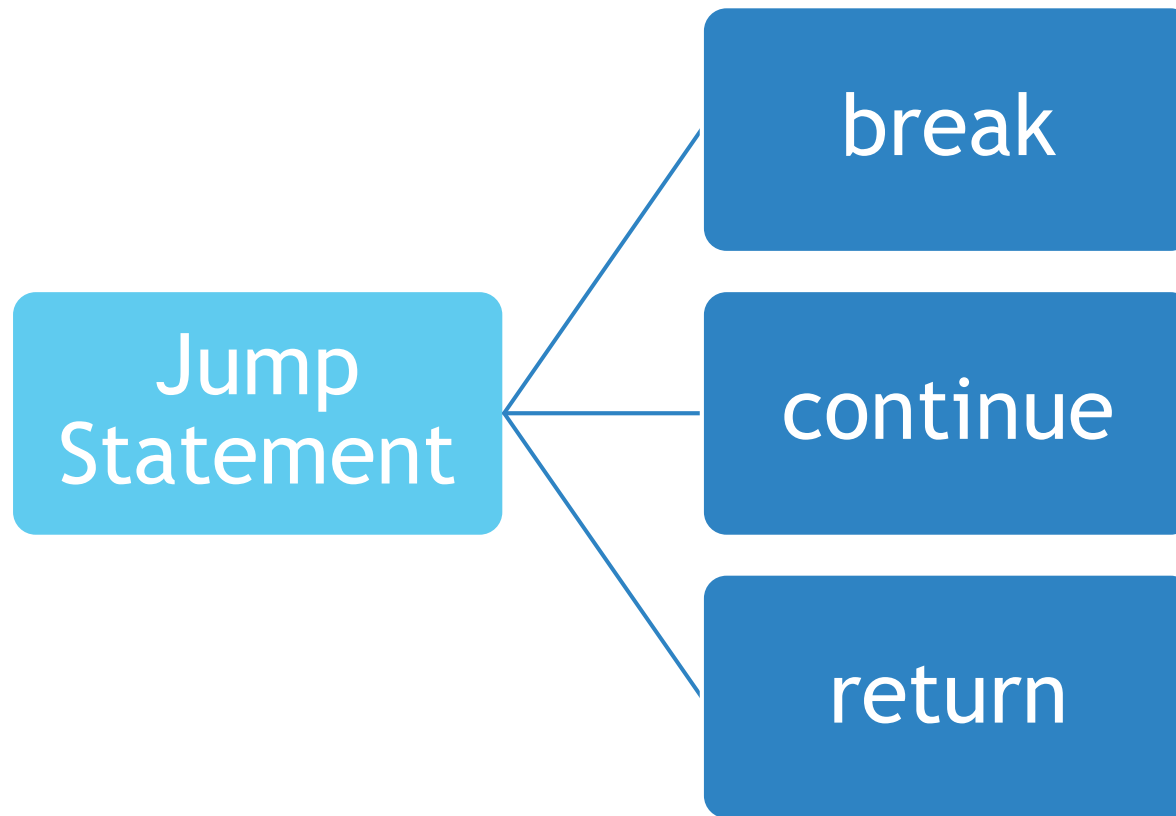
**statements;**

**}**

- ▶ **Purpose:** To evaluate the statements from initial value to final value with given step.

```
for(int i=0; i<10; i++)  
{  
    System.out.println(i);  
}
```

# Jump Statements



# The break statement

- ▶ This statement is used to jump out of a loop.
- ▶ Break statement was previously used in switch - case statements.
- ▶ On encountering a break statement within a loop, the execution continues with the next statement outside the loop.
- ▶ The remaining statements which are after the break and within the loop are skipped.
- ▶ Break statement can also be used with the label of a statement.
- ▶ A statement can be labeled as follows.

**statementName : SomeJavaStatement**

When we use break statement along with label as:

**break statementName;**

# Example

```
class BreakLabelDemo {  
    public static void main(String args[]) {  
        outer: for (int i = 0; i < 3; i++) {  
            System.out.print("Pass " + i + ": ");  
            for (int j = 0; j < 100; j++) {  
                if (j == 10)  
                    break outer; // exit both loops  
                System.out.print(j + " ");  
            }  
            System.out.println("This will not print");  
        }  
        System.out.println("Loops complete.");  
    }  
}
```

# continue Statement

- ▶ This statement is used only within looping statements.
- ▶ When the continue statement is encountered, the next iteration starts.
- ▶ The remaining statements in the loop are skipped. The execution starts from the top of loop again.



# Example

```
class ContinueLabel {  
    public static void main(String args[]) {  
outer: for (int i=0; i<10; i++) {  
        for(int j=0; j<10; j++) {  
            if(j > i) {  
                System.out.println();  
                continue outer;  
            }  
            System.out.print(" " + (i * j));  
        }  
    }  
    System.out.println();  
}  
}
```

# String Handling

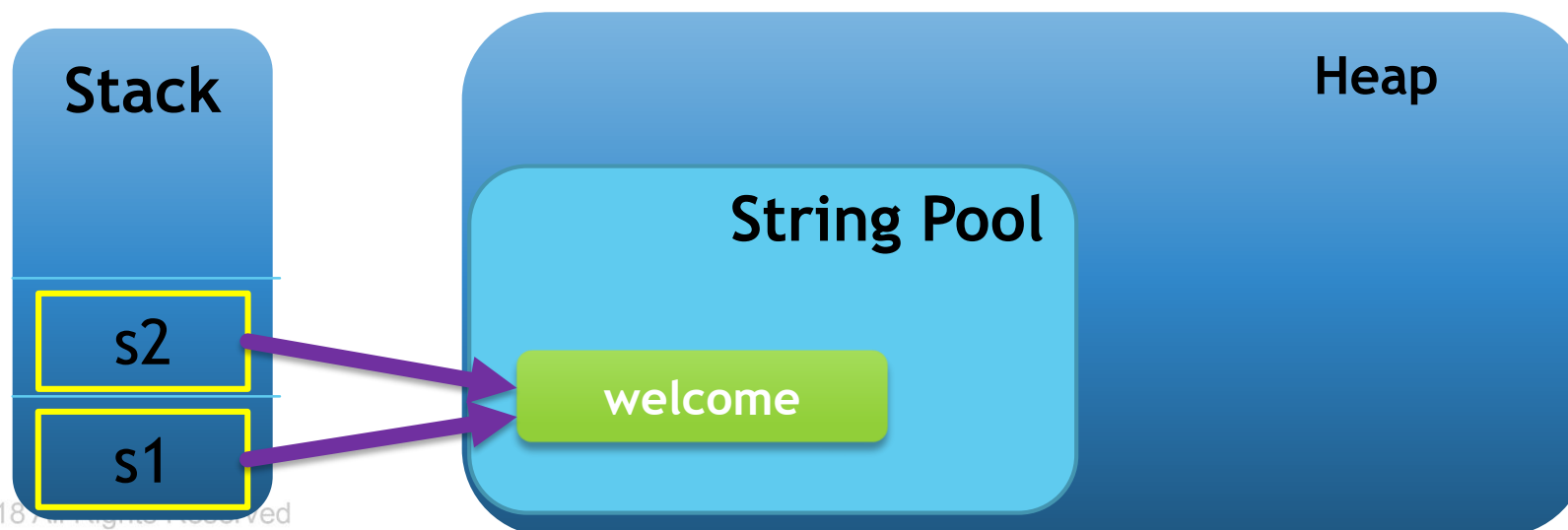


# The String Class (`java.lang.String`)

- ▶ String is a sequence of characters placed in double quote(" ").
- ▶ Strings are constant , their values cannot be changed in the same memory after they are created.

# How to create String object

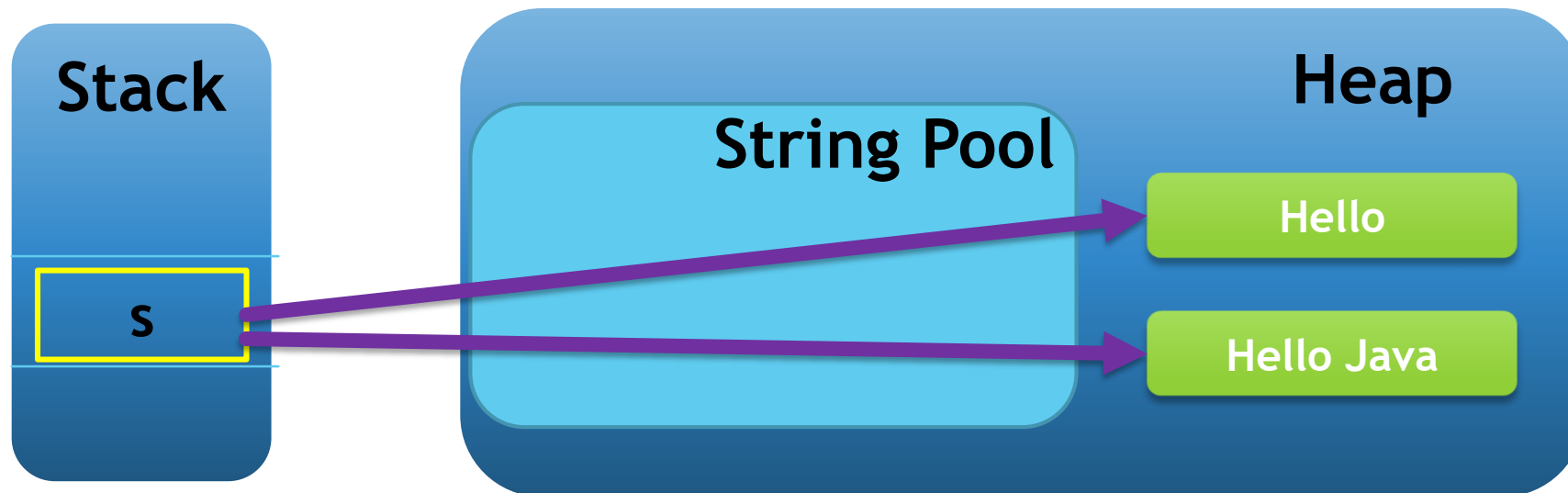
- ▶ There are two ways to create String object:
  - ▶ By string literal.
  - ▶ By new keyword.
- ▶ By string literal:
  - ▶ For Example:
    - ▶ `String s1="welcome"; String s2="welcome";` //no new object will be created



## Create String By new keyword:-

### ► For Example:

```
String s=new String("Hello");  
s=new String("Hello Java");
```



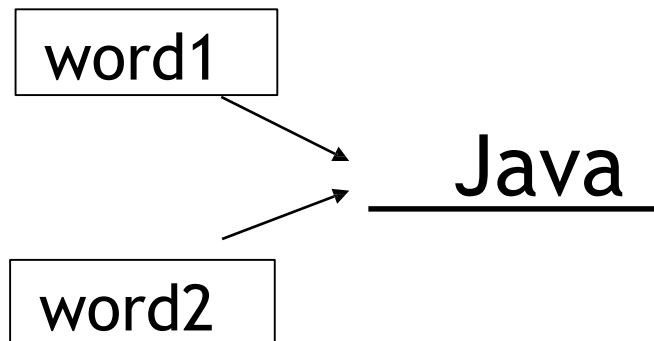
# Immutability

- In java, string objects are immutable.
- Immutable simply means unmodifiable or unchangeable.
- Once string object is created its data or state can't be changed but a new string object is created.

# Advantages Of Immutability

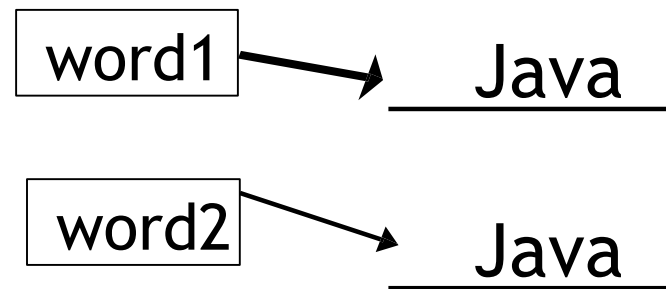
Use less memory:

```
String word1 = "Java";  
String word2 = word1;
```



more efficient: saves  
memory

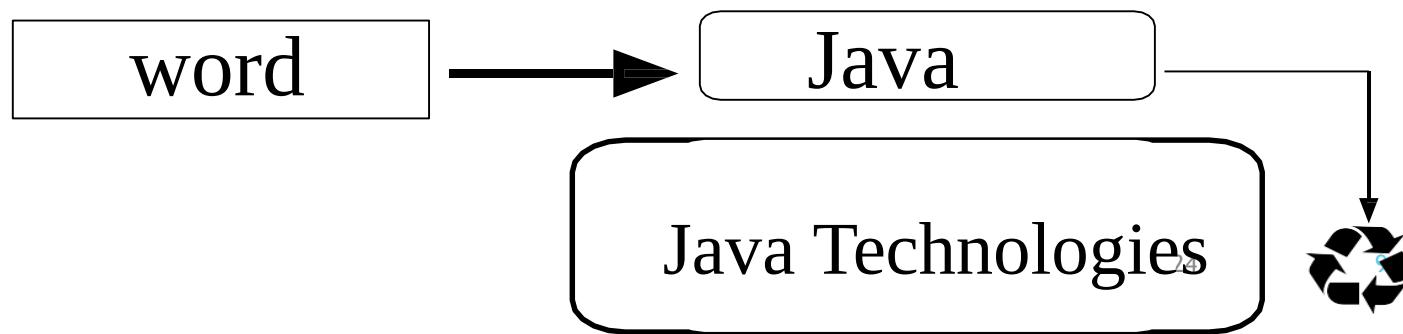
```
String word1 = "Java";  
String word2 = new String("Java");
```



Less efficient: wastes  
memory

# Disadvantages of Immutability

- **Less efficient** — you need to create a new string and throw away the old one even for small changes.
- **Example:**
- String word = “Java”;
- word= word.concat(“Technologies”);





# Constructors

- ▶ No Argument Constructors:
- ▶ No-argument constructor creates an empty String. Rarely used.
- ▶ Example: `String empty=new String();`
- ▶ Other Constructors:
- ▶ Most other constructors take an array as a parameter to create a String.
- ▶ Example:
- ▶ `char[] letters = {'J', 'a', 'v', 'a'};`
- ▶ `String word = new String(letters); // "Java"`

# String Methods

- **substring(int begin):**
  - Returns substring from begin index to end of the String.  
Example: `String s="helping";`  
`System.out.println(s.substring(2));`//lping
- **equals():**
  - To perform content comparison where case is important.
    - Example:  
`String s="java";`  
`System.out.println(s.equals("java"));`//true

- **concat():** Adding two strings we use this method

Example:

```
String s="test";  
s= s.concat("software");  
System.out.println(s);  
  
// testsoftware
```

## length() , charAt()

**int length();**

- Returns the number of characters in the string
- Returns the char at position i.

**char charAt(i);**

Character positions in strings are numbered starting from 0 - just like arrays.

Returns:

`"Problem".length();`

`"Window".charAt (2);`



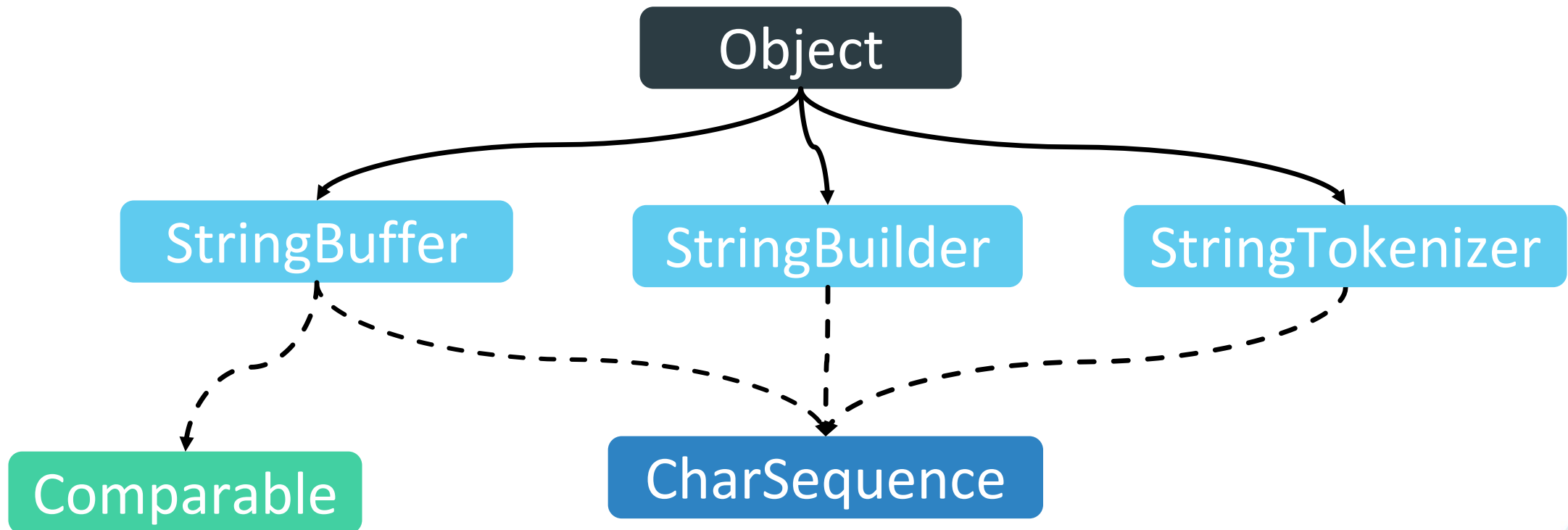
# New features of String

- In JDK 1.7 (Java 7):
  - Java 7 extended the capability of switch case to use Strings also, earlier java versions doesn't support this.
  - ▶ If you are implementing conditional flow for Strings, you can use if-else conditions and you can use switch case if you are using Java 7 or higher versions.

# StringBuffer, StringBuilder and StringTokenizer

- **Limitation of String in Java ?**
- **What is mutable string?**
  - A string that can be modified or changed is known as mutable string. StringBuffer and StringBuilder classes are used for creating mutable string.
- **Differences between String and StringBuffer in java?**
  - Main difference between String and StringBuffer is String is immutable while StringBuffer is mutable

# Relations between StringBuffer, StringBuilder and StringTokenizer:



# StringBuffer

- StringBuffer is synchronized and allows us to mutate the string.
- StringBuffer has many utility methods to manipulate the string.
- This is more useful when using in a multithreaded environment.
- Always modified in same memory location.

# StringBuffer

## ➤ **StringBuffer Constructors:**

- `public StringBuffer()`
- `public StringBuffer(int capacity)`
- `public StringBuffer(String s)`
- `public StringBuffer(CharSequence cs)`



# Method Use In StringBuffer

- append
- insert
- delete
- reverse
- Replacing Character at given index

# StringBuilder

- StringBuilder is the same as the StringBuffer class
- The StringBuilder class is not synchronized and hence in a single threaded environment, the overhead is less than using a StringBuffer.

# StringTokenizer

- A *token* is a portion of a string that is separated from another portion of that string by one or more chosen characters (called *delimiters*).
- The *StringTokenizer* class contained in the *java.util package* can be used to break a string into separate tokens.
- This is particularly useful in those situations in which we want to read and process one token at a time;



# Lab Exercises

# Lab Exercise

➤ Split a string represent an IP address:

➤ Use string split("\\.")

➤ Use StringTokenizer

➤ The program, for example: your Input

➤ 163.121.12.30

➤ The result is :

163

121

12

30

```
public class IPCutter {

    String cmdLine;

    public IPCutter(String cmdLine) { ...3 lines }
    int[] doIPSplit() { ...17 lines }
```

```
public class Lecture_Demo {

    public static void main(String[] args) {

        String commandLine="163.121.12.30";
        IPCutter cut=new IPCutter( cmdLine:commandLine);
        System.out.println("The output of "+ commandLine+ " is");

        int[] out=cut.doIPSplit();

        for(int i=0;i<out.length;i++)
            System.out.println(out[i]);
```

# Lab Exercise

- Given a sentence and a word, your task is that to count the number of occurrences of the given word in the string and print the number of occurrence of the word.
- Perform the above task using only methods of the String class (2 ways).

# Lab Exercise

- Develop an application that extracts the minimum and maximum of the elements of an array of 1000 element and compute the search running time.
- Develop an application to implement the binary search algorithm and compute the search running time.
- Hint: Use `System.currentTimeMillis()` or `System.nanoTime()`.

```
public class ArrayAlgorithms {  
  
    public int max(int[] array) { ...14 lines }  
  
    public int min(int[] array) { ...12 lines }  
  
}
```

```
public class ArrayMain {  
  
    public static void main(String[] args) {  
        int[] myArray = {23, 92, 56, 39, 93, 80, 123, 152, 70, 60, 90, 5, 88, 66, 77, 33};  
        ArrayAlgorithms m = new ArrayAlgorithms();  
        System.out.println("Maximum value in the array is::" + m.max( array:myArray));  
        System.out.println("Minimum value in the array is::" + m.min( array:myArray));  
    }  
  
}
```